ST2MLE : Machine Learning for IT Engineers

LAB 2 : Unsupervised learning and text classification

Part 1:

- Exploratory data analysis (EDA)
- Data pre-processing
- Clustering (k-means)
- Dimensionality reduction (PCA)

Part 2:

- Text pre-processing
- Text vectorization
- Text classification

Part 1: EDA, Clustering and Dimensionality reduction

Context: Pima Indians Diabetes Dataset

The **Pima Indians Diabetes Dataset** contains medical diagnostic measurements of **female Pima Indian patients aged 21 and older**.

The dataset was collected by the **National Institute of Diabetes and Digestive and Kidney Diseases (NIDDK)**, USA, to explore risk factors associated with the development of **Type 2 diabetes**, which is a common chronic disease within the Pima Indian community.

While the original task is to predict diabetes diagnosis (binary classification), in this lab, we will treat the dataset as **unlabeled** and explore it using **unsupervised techniques**, such as:

- **Exploratory Data Analysis (EDA)**
- **K-Means Clustering**
- **Principal Component Analysis (PCA)**

Objectives

At the end of Part 1 of the lab, you will be able to:

- Perform **EDA on numeric datasets**
- Check **missing data, outliers, scaling**
- Apply **K-Means clustering** and interpret clusters
- Apply **PCA for dimensionality reduction and visualization**
- Compare the clustering results using original features vs. reduced features

You can download the Pima Indians dataset from Openml:
https://www.openml.org/search?type=data&status=active&id=43582

Or use openml.datasets.get_dataset(id) # id is the id of the dataset.

Dataset information:

Feature	Description
Pregnancies	Number of times pregnant
Glucose	Plasma glucose concentration
BloodPressure	Diastolic blood pressure (mm Hg)
SkinThickness	Triceps skin fold thickness (mm)
Insulin	2-Hour serum insulin (mu U/ml)
BMI	Body mass index (weight in kg/(height in m)^2)
DiabetesPedigreeFunction	Diabetes pedigree function: a person's genetic predisposition to diabetes
Age	Age in years

Note: The dataset contains **8 numerical features and 768 instances**. For the purpose of this lab, ignore the Outcome (target) column.

Exercise 1: Exploratory Data Analysis (EDA)

1. Load the dataset using pandas.
2. Display basic statistics (describe()) for all features.
3. Visualize the data distribution of each feature (histograms, boxplots).
4. Check for missing data or anomalous zeros (e.g., Insulin and SkinThickness have zeros which are unrealistic).
5. Identify potential outliers using boxplots.
6. Standardize the data (use StandardScaler). Pay attention to potential data leakage when standardizing features.

Exercise 2: K-Means Clustering

1. Apply **K-Means clustering** (with k=2) to the **standardized data**.
2. Visualize the resulting clusters using a scatter plot (select two features for visualization).
3. Interpret the results (do the clusters make sense?).

Exercise 3: PCA and Visualization

1. Apply **PCA** to reduce the dataset to **2 components**.
2. Display the **explained variance ratio** for each principal component
3. Plot the data points using the **first 2 principal components**.
4. Color the points by the **K-Means cluster labels** from Exercise 2.
5. Which features contribute most to **PC1** and **PC2**?
6. Inspect the PCA loadings (components) and interpret which original features have the highest weights.

Exercise 4: K-Means on PCA-transformed data

1. Apply **K-Means clustering again**, but now **on the 2D PCA-transformed data**.
2. Compare the clusters obtained in Exercise 2 vs. Exercise 4.
3. Discuss the advantages or drawbacks of clustering in the **original space vs. PCA space**.

Discussion

- When does **PCA help clustering?**
- How did **scaling** impact the results?
- Are clusters meaningful in this context? How could you improve them?

Part 2: Text classification using Bag of Words, TF-IDF, Word2Vec and BERT embeddings.

Context: News classification

The **20 Newsgroups dataset** contains a collection of approximately **20,000 newsgroup documents**, organized into **20 different newsgroups**. Many of these newsgroups focus on topics related to **computing**, such as **comp.programming**, **comp.os.ms-windows.misc**, **comp.sys.ibm.pc.hardware**, **comp.sys.mac.hardware** and **comp.windows.x**, which include discussions around computer hardware, operating systems, and software development. This variety of newsgroups makes the dataset relevant for exploring a wide range of **IT-related issues** including **IT infrastructure**, **system administration**, **networking**, and **programming practices**. These categories provide a rich set of topics for text classification tasks in the computing domain.

Dataset Information

- **Features:** The dataset contains text data, with documents from newsgroups related to topics such as **software engineering, programming**, and general discussions.
- **Target:** The target variable is the **newsgroup category**, representing the class label for each document.

Objectives

In this Part 2 of the lab, you will:

- **Preprocess the text** data using various techniques.
- Convert the text into **vector representations** using: **Bag of Words (BoW), TF-IDF**, **Word2Vec and BERT embeddings**
- Apply **Naive Bayes** and **Logistic Regression** models to classify the newsgroups.
- Compare the performance of each method.

Exercise 1: Text Preprocessing

1. **Load the dataset** using scikit-learn's fetch_20newsgroups() method.
2. **Filter the dataset** to keep only the records labeled with the computing-related categories:

    1. comp.graphics
    2. comp.os.ms-windows.misc
    3. comp.sys.ibm.pc.hardware
    4. comp.sys.mac.hardware
    5. comp.windows**.x**

3. **Preprocess the text**:
    - Convert the text to **lowercase**.
    - **Remove punctuation** (use regular expressions).
    - **Lemmatize** the words using spaCy.
4. **Split the dataset** into training and test sets (e.g., 80% train, 20% test). Any potential leakage?

Exercise 2: Bag of Words (BoW)

1. **Vectorize the text** using **Bag of Words**.
    - Use CountVectorizer from **scikit-learn**. **Remove stopwords.**
    - Visualize the size of the vocabulary (number of unique words).
2. **Train a model** using **Naive Bayes** (MultinomialNB) or **Logistic Regression**.
3. **Evaluate the performance** of the model using **accuracy** on the test set.

Exercise 3: TF-IDF

1. **Vectorize the text** using **TF-IDF** (Term Frequency-Inverse Document Frequency).
    - Use TfidfVectorizer from **scikit-learn**. **Remove stopwords.**
    - Compare the average **TF-IDF values** for the top 10 frequent terms.
2. **Train a model** using **Naive Bayes** or **Logistic Regression** (use the same model you used in exercice 2 for consistency).
3. **Evaluate the performance** of the model using **accuracy** on the test set.
4. **Question**: How does TF-IDF improve upon the Bag of Words model in terms of handling the importance of terms?

Exercise 4: Word2Vec

1. **Vectorize the text** using **Word2Vec** embeddings.
    - Use the Word2Vec model from **Gensim** to train word embeddings.
    - Use the average of the word embeddings to represent each document.
2. **Train a model** using **Naive Bayes** (GaussianNB not MultinomialNB) or **Logistic Regression**.
3. **Evaluate the performance** of the model using **accuracy** on the test set.
4. **Question**: How does Word2Vec perform compared to Bag of Words and TF-IDF? What are the key differences?

Exercise 5: BERT Embeddings

1. **Vectorize the text** using **BERT embeddings**.
    - Use **Hugging Face's transformers library** to obtain BERT embeddings for each document.
    - Consider using a **pre-trained BERT model** (e.g., bert-base-uncased).
    - Apply **mean pooling** over token embeddings to get a document representation.
2. **Train a model** using **Naive Bayes** or **Logistic Regression**.
3. **Evaluate the performance** of the model using **accuracy** on the test set.
4. **Question**: How does BERT's contextual embeddings compare to the other vectorization techniques? Is it a better representation of the text?

Summary: Comparison of Models and Vectorization Techniques

1. **Compare** the **accuracy** of all models (Naive Bayes, Logistic Regression) trained with the different vectorization techniques: Bag of Words, TF-IDF, Word2Vec, Doc2Vec and BERT embeddings.
2. **Plot a bar chart** comparing the accuracy of each method.
3. **Questions**:
    - Which vectorization method performs the best?
    - What trade-offs do you observe in terms of model complexity and accuracy?
    - Is BERT significantly better than the other methods?

Deliverables (to be submitted on Moodle)

- A Jupyter notebook or `.py` file with completed tasks. The jupyter notebook should include screenshots of the above experiments and a final comparison table with analyses. 2 to 3 students can work together on the same lab and submit only one report/notebook.